

AFRILANG Stdlib

std/c/parsepath

Français. Bibliothèque AFRILANG 'std/c/parsepath' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : pathLen, pathUpper, pathLower, pathTrim, pathSplit, pathJoin, pathReplace.

```
'import "std/c/parsepath"' · 'use parsepath'
```

```
- 'pathLen(s text) → number' - 'pathUpper(s text) → text' - 'pathLower(s text) → text' - 'pathTrim(s text) → text' - 'pathSplit(s text, delim text) → list text' - 'pathJoin(parts list text, delim text) → text' - 'pathReplace(s text, from text, to text) → text'
```

English. AFRILANG library 'std/c/parsepath' (tier complex, category complex). Exports 7 function(s): pathLen, pathUpper, pathLower, pathTrim, pathSplit, pathJoin, pathReplace.

```
'import "std/c/parsepath"' · 'use parsepath'
```

```
- 'pathLen(s text) → number' - 'pathUpper(s text) → text' - 'pathLower(s text) → text' - 'pathTrim(s text) → text' - 'pathSplit(s text, delim text) → list text' - 'pathJoin(parts list text, delim text) → text' - 'pathReplace(s text, from text, to text) → text'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/parsepath' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : pathLen, pathUpper, pathLower, pathTrim, pathSplit, pathJoin, pathReplace.

'import "std/c/parsepath"' · 'use parsepath'

```
- `pathLen(s text) → number`
- `pathUpper(s text) → text`
- `pathLower(s text) → text`
- `pathTrim(s text) → text`
- `pathSplit(s text, delim text) → list text`
- `pathJoin(parts list text, delim text) → text`
- `pathReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parsepath"
use parsepath
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- pathLen(s text) → number•
pathUpper(s text) → text•
pathLower(s text) → text•
pathTrim(s text) → text•
pathSplit(s text, delim text) → list•
pathJoin(parts list text, delim text) → text•
pathReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/parsepath"
use parsepath
```

```
create result = pathLen(/* args */)
say result
```

```
create result = pathUpper(/* args */)
say result
```

```
create result = pathLower(/* args */)
say result
```

```
create result = pathTrim(/* args */)
say result
```

```
create result = pathSplit(/* args */)
say result
```

```
create result = pathJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/parsepath"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module parsepath

```
function pathLen(s text) returns number
end
```

```
function pathUpper(s text) returns text
end
```

```
function pathLower(s text) returns text
end
```

```
function pathTrim(s text) returns text
end
```

```
function pathSplit(s text, delim text) returns list text
end
```

```
function pathJoin(parts list text, delim text) returns text
end
```

```
function pathReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/parsepath' (tier complex, category complex). Exports 7 function(s): pathLen, pathUpper, pathLower, pathTrim, pathSplit, pathJoin, pathReplace.

```
'import "std/c/parsepath"' · 'use parsepath'
```

```
- `pathLen(s text) → number`
- `pathUpper(s text) → text`
- `pathLower(s text) → text`
- `pathTrim(s text) → text`
- `pathSplit(s text, delim text) → list text`
- `pathJoin(parts list text, delim text) → text`
- `pathReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parsepath"
use parsepath
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- pathLen(s text) → number•
pathUpper(s text) → text•
pathLower(s text) → text•
pathTrim(s text) → text•
pathSplit(s text, delim text) → list•
pathJoin(parts list text, delim text) → text•
pathReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/parsepath"
use parsepath
```

```
create result = pathLen(/* args */)
say result
```

```
create result = pathUpper(/* args */)
say result
```

```
create result = pathLower(/* args */)
say result
```

```
create result = pathTrim(/* args */)
say result
```

```
create result = pathSplit(/* args */)
say result
```

```
create result = pathJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/parsepath"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module parsepath

```
function pathLen(s text) returns number
end
```

```
function pathUpper(s text) returns text
end
```

```
function pathLower(s text) returns text
end
```

```
function pathTrim(s text) returns text
end
```

```
function pathSplit(s text, delim text) returns list text
end
```

```
function pathJoin(parts list text, delim text) returns text
end
```

```
function pathReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/parsepath"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/parsepath/>

Source (excerpt)

```
module parsepath

function pathLen(s text) returns number
end

function pathUpper(s text) returns text
end

function pathLower(s text) returns text
end

function pathTrim(s text) returns text
end

function pathSplit(s text, delim text) returns list text
end

function pathJoin(parts list text, delim text) returns text
end

function pathReplace(s text, from text, to text) returns text
end
end
```